

Scheduling

Priority --- Fixed---Rate Monotonic and Deadline
 ---- Dynamic---Earliest deadline-Least Slack time

Timing requirements on processes

- **Period**: interval between process activations.
- **Initiation interval**: reciprocal of period.
- **Initiation time**: time at which process becomes ready.
- **Deadline**: time at which process must finish.

Timing violations

- What happens if a process doesn't finish by its deadline?
 - **Hard deadline**: system fails if missed.
 - **Soft deadline**: user may notice, but system doesn't necessarily fail.

In a scheduling algorithm, the key timings include Arrival Time (when a process enters the ready queue), Burst Time (time needed to execute a process), Completion Time (when a process finishes execution), Turnaround Time (time between arrival and completion), and Waiting Time (time spent in the ready queue waiting for the CPU).

- **Arrival Time:** This is the moment a process becomes ready to be executed and enters the ready queue.
- **Burst Time:** Represents the actual processing time required by a process on the CPU.
- **Completion Time:** The time at which a process finishes executing and leaves the system.
- **Turnaround Time:** Calculated as the difference between Completion Time and Arrival Time, indicating the total time a process spends in the system.
• ***Turn Around Time = Completion Time – Arrival Time***
- **Waiting Time:** The time a process spends in the ready queue waiting for its turn to be executed on the CPU.
• ***Waiting Time = Turn Around Time – Burst Time***
- **Response time:** The time spent waiting for the CPU for the first time
- **Release time of a job:** It is the time at which the job becomes ready for execution.
- **Execution time of a job:** It is the time taken by the job to finish its execution
- **Slack time and Scheduling overhead** means The time taken by dispatcher to move the process from ready state to running

Scheduling metrics

- How do we evaluate a scheduling policy:
 - Ability to satisfy all deadlines.
 - CPU utilization---percentage of time devoted to useful work.
 - Scheduling overhead---time required to make scheduling decision.

- **Earliest deadline first (EDF)** comes under the category of the dynamic scheduling algorithm.
- We can use it in real-time operating systems for scheduling tasks as well as processes with specific deadlines.
- **It's a priority-based algorithm where we assign the highest priority to the tasks with the earliest deadline.**
- Additionally, the CPU executes the scheduled tasks based on priority.
- The algorithm assigns a priority to each task based on its deadline and chooses the task with the earliest deadline for execution.
- Additionally, if multiple tasks have the same deadline, **the CPU executes the task with the highest priority.**
- **EDF is an optimal algorithm for scheduling a set of periodic tasks with known deadlines** and execution times as long as the total utilization of the tasks doesn't exceed unity.
- Hence, the total execution time of all tasks should be less than the total available time

Hyperperiod

Hyperperiod: The hyper period of a set of periodic tasks is the least common multiple (LCM) of their periods.

- It is the smallest time after which the **pattern of job release times for all tasks in the set repeats.**
- Must look at the hyperperiod schedule to find all task interactions.
- Hyperperiod can be very long if task periods are not chosen carefully.

The hyper period is an important parameter in real-time scheduling, as it determines the scheduling cycle for the tasks.

The hyper-period is the smallest interval of time after which the periodic patterns of all the tasks is repeated.

Hyperperiod example

Long hyperperiod:

P1 7 ms.

P2 11 ms.

P3 15 ms.

LCM = 1155 ms.

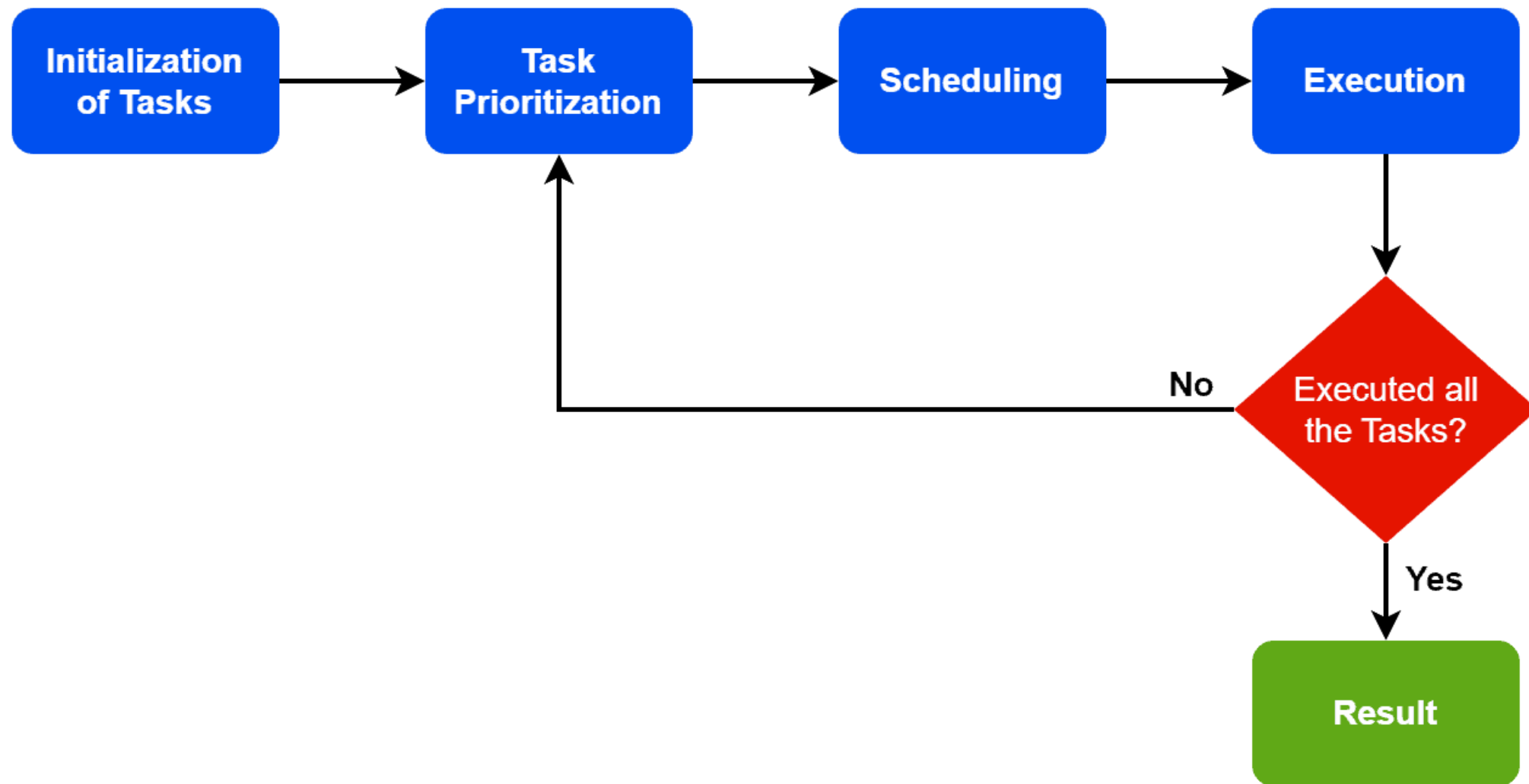
Shorter hyperperiod:

P1 8 ms.

P2 12 ms.

P3 16 ms.

LCM = 96 ms.



- **The earliest deadline first scheduling algorithm consists of four steps: initialization, task prioritization, task scheduling, and task execution.**
- The first step is to initialize the available tasks. Additionally, along with initialization, we assign each task a deadline based on completion requirements. The next step is to assign priority to each task. The system sorts the tasks in order of their deadlines, with the task having the earliest deadline assigned the highest priority.
- Furthermore, the system selects the task with the **earliest deadline for execution. If multiple tasks have the same deadline, we select the task with the highest priority.**
- Finally, **the CPU executes the selected task until completed or the deadline is reached. If the task is completed before its deadline, the system returns to the task prioritization step to select the next task for execution. If the deadline is reached, the task is considered to be missed.**

- Now before the algorithm terminates, we need to check two conditions.
- First, we need to check whether **all the tasks are executed.**
- Additionally, we **check if all deadlines have been missed.** If one of these conditions is met, we need to go back to the task prioritization step and repeat the steps until the algorithm terminates.
- EDF is a dynamic scheduling algorithm where the task priority can change over time as deadlines approach.
- Hence, **the algorithm continually updates the priority of tasks based on their deadlines and selects the task with the earliest deadline for execution.**

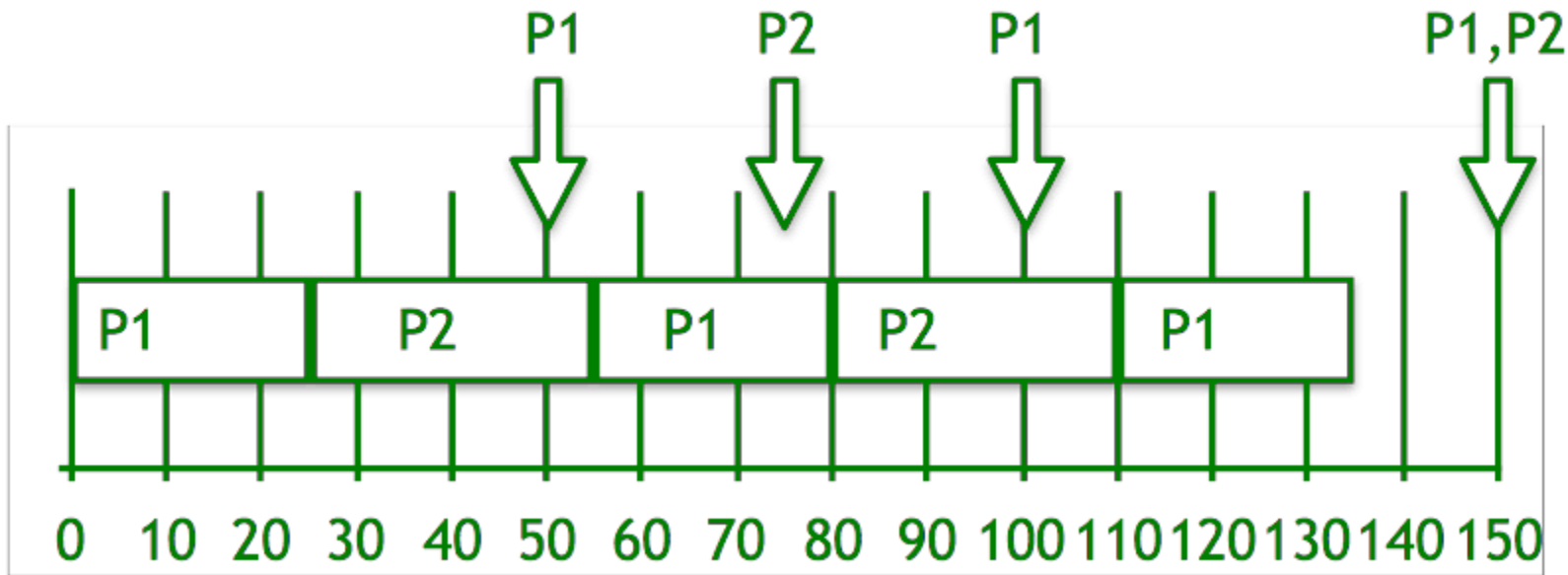
Consider two processes P1 and P2.

Let the period of P1 be $p_1 = 50$

Let the processing time of P1 be $t_1 = 25$

Let the period of P2 be $period_2 = 75$

Let the processing time of P2 be $t_2 = 30$



If deadline not given in EDF then period will be considered as Deadline

1. Deadline of P1 is earlier, so priority of $P1 > P2$.

2. Initially P1 runs and completes its execution of 25 time.

3. After 25 times, P2 starts to execute until 50 times, when P1 is able to execute.

4. Now, comparing the deadline of (P1, P2) = (100, 75), P2 continues to execute.

5. P2 completes its processing at time 55.

6. P1 starts to execute until time 75, when P2 is able to execute.

7. Now, again comparing the deadline of (P1, P2) = (100, 150), P1 continues to execute.

8. Repeat the above steps...

9. Finally at time 150, both P1 and P2 have the same deadline, so P2 will continue to execute till its processing time (already P1 completed between 50 to 100 after P2, P1 starts to execute).

Schedulability bound for periodic/sporadic tasks

Theorem

Given a task set of periodic or sporadic tasks, with relative deadlines equal to periods, the task set is schedulable by EDF if and only if

$$U = \sum_{i=1}^N \frac{C_i}{T_i} \leq 1$$

Corollary

EDF is an optimal algorithm, in the sense that if a task set is schedulable, then it is schedulable by EDF.

Proof.

In fact, if $U > 1$ no algorithm can successfully schedule the task set; if $U \leq 1$, then the task set is schedulable by EDF (and maybe by other algorithms). □

➤ 1. Hyper period

- 2. With scheduling periodic processes that have deadlines equal to their periods, EDF has a utilization bound of 100%. Thus, **the schedulability test for EDF** is:

$$U = \sum_{i=1}^n \frac{C_i}{T_i} \leq 1,$$

where the $\{C_i\}$ are the worst-case computation-times of the n processes and the $\{T_i\}$ are their respective inter-arrival periods (assumed to be equal to the relative deadlines).

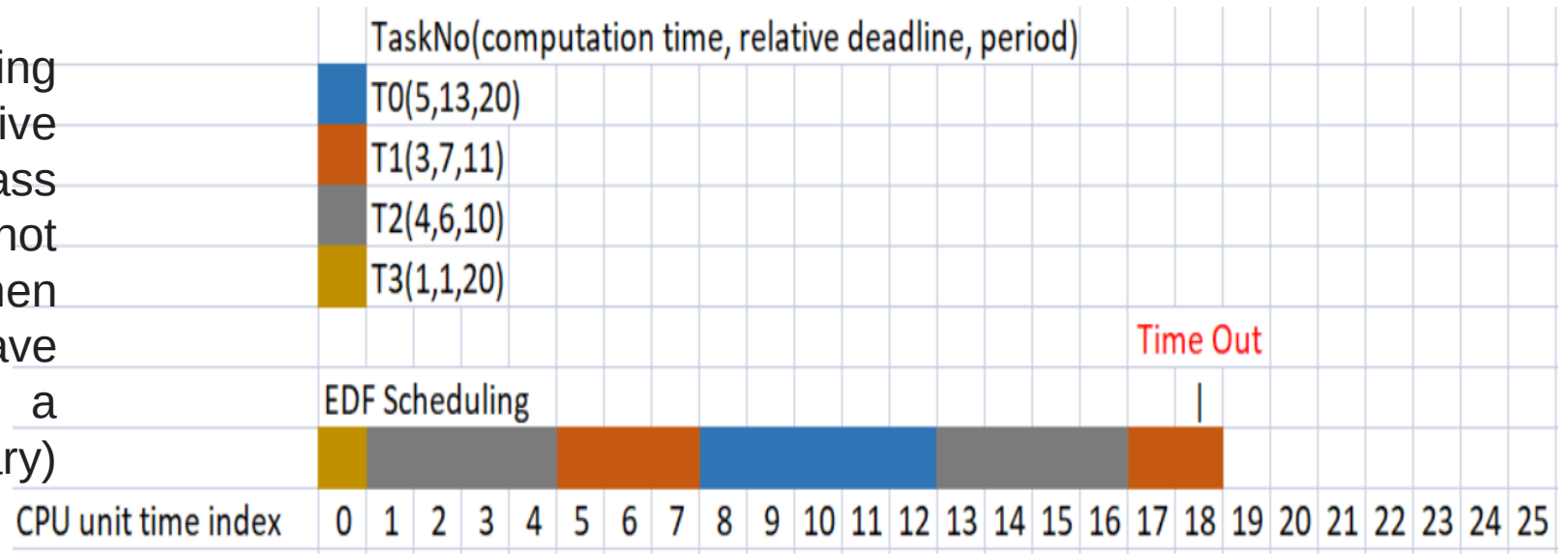
That is, EDF can guarantee that all deadlines are met provided that the total [CPU](#) utilization is not more than 100%. Compared to fixed-priority scheduling techniques like [rate-monotonic scheduling](#), EDF can guarantee all the deadlines in the system at higher loading.

Note that use the schedulability test formula under deadline as period. When deadline is less than period, things are different. Here is an example: The four periodic tasks needs scheduling, where each task is depicted as TaskNo(computation time, relative deadline, period). They are T0(5,13,20), T1(3,7,11), T2(4,6,10) and T3(1,1,20). This task group meets utilization is no greater than 1.0, where utilization is calculated as $5/20+3/11+4/10+1/20 = 0.97$ (two digits rounded), but it's still unscheduable, check **EDF Scheduling Failure** figure for details.

EDF is also an *optimal* scheduling algorithm on non-preemptive uniprocessors, but only among the class of scheduling algorithms that do not allow inserted idle time. When scheduling periodic processes that have deadlines equal to their periods, a sufficient (but not necessary) schedulability test for EDF becomes:

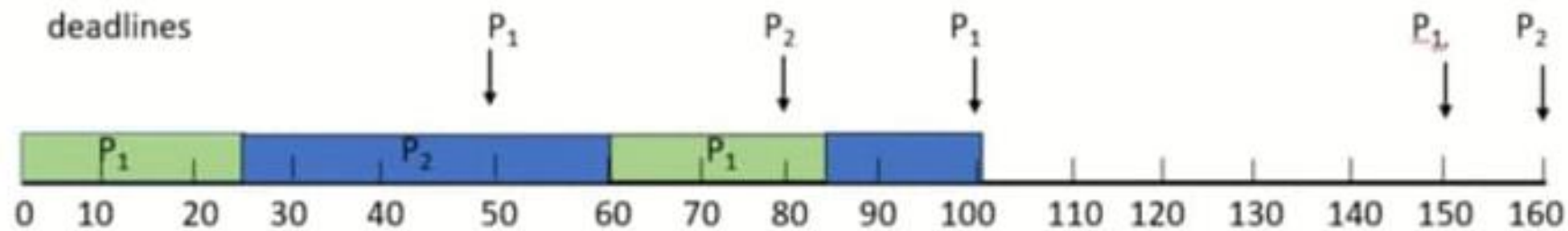
$$U = \sum_{i=1}^n \frac{C_i}{T_i} \leq 1 - p,$$

Where p represents the penalty for non-preemption, given by $\max \{C_i\} / \min \{T_i\}$. If this factor can be kept small, non-preemptive EDF can be beneficial as it has low implementation overhead.



Earliest Deadline First Scheduling (EDF)

- Priorities are assigned according to deadlines:
 - Earlier the deadline, higher the priority
 - $p_1 = 50$, $t_1 = 25$, $p_2 = 80$, $t_2 = 35$
 - Preemption is not done if deadline is later



Earliest-Deadline-First Scheduling

- ① • Dynamic Priority
- ② • Highest priority to lowest deadline.

Process	Execution Time	Period
P1	25	50
P2	30	75

Step 1: Calculate Hyperperiod

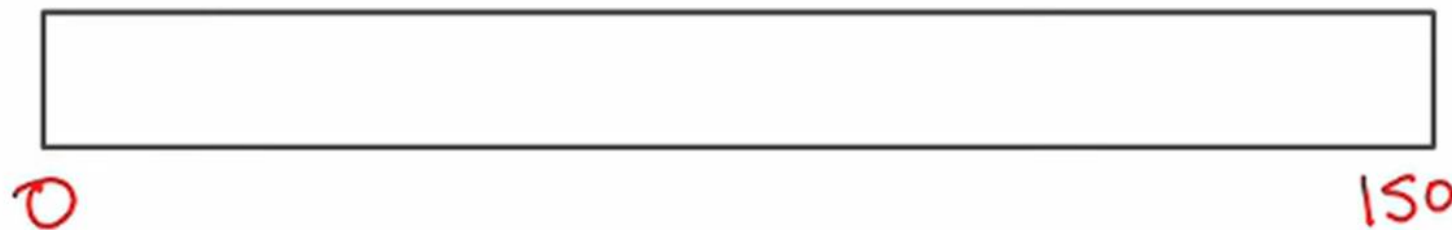
$$H = \text{LCM}(50, 75) \\ = 150$$

Step 2: Calculate number of runs for each process

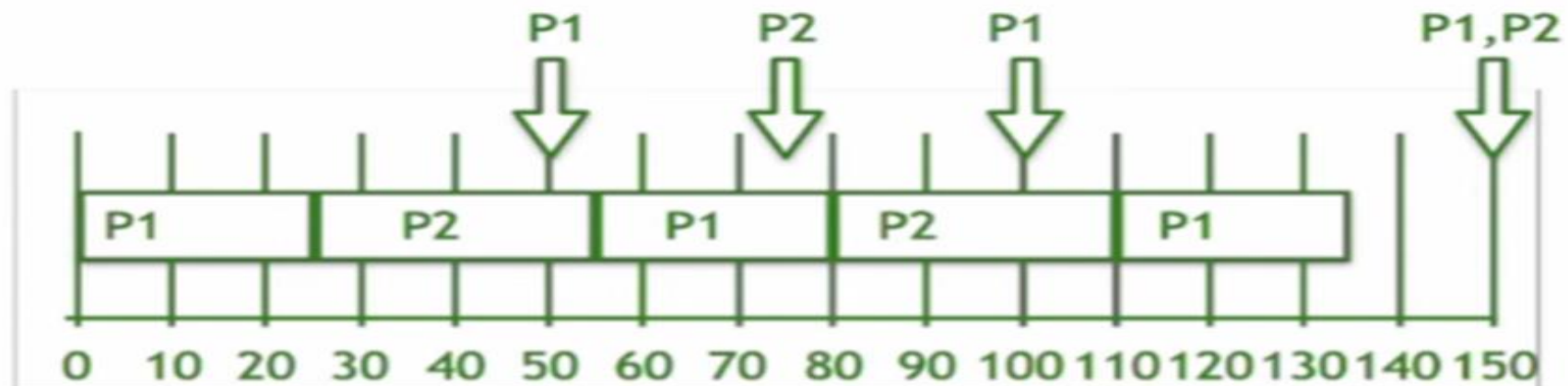
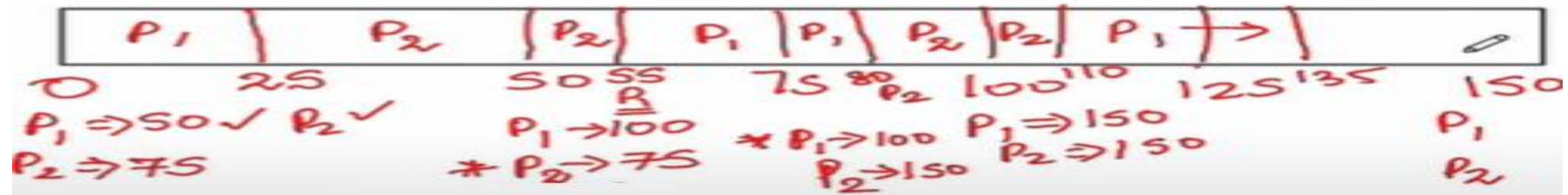
$$P_1 \rightarrow 150/50 = 3$$

$$P_2 \rightarrow 150/75 = 2$$

Step 3: Gantt Chart



Step 3: Gantt Chart



Steps for solution:

1. Deadline of P1 is earlier, so priority of $P1 > P2$.
2. Initially P1 runs and completes its execution of 25 time.
3. After 25 times, P2 starts to execute until 50 times, when P1 is able to execute.
4. Now, comparing the deadline of $(P1, P2) = (100, 75)$, P2 continues to execute.
5. P2 completes its processing at time 55.
6. P1 starts to execute until time 75, when P2 is able to execute.
7. Now, again comparing the deadline of $(P1, P2) = (100, 150)$, P1 continues to execute.
8. Repeat the above steps...
9. Finally at time 150, both P1 and P2 have the same deadline, so P2 will continue to execute till its processing time after which P1 starts to execute.

Real-Time CPU Scheduling. Consider the following single-thread process, executing times, deadlines and periods.

Draw the corresponding Gantt charts depicting the scheduling procedures for these processes using Rate-Monotonic (RM) scheduling and Earliest Deadline First (EDF) scheduling with in the first 20 timeslots. Assume all processes arrive at timeslot 0.

Process	Burst Time	Deadline	Period
P1	3	7	20
P2	2	4	5
P3	2	8	10

$U = C1/P1 + C2/P2 + C3/P3 = (3/20) + (2/5) + (2/10) = 0.75 = 75\%$
 $U = 75 < 100\%$, which indicates that the processes are schedulable.

$$U = \sum_{i=1}^n \frac{C_i}{T_i} \leq 1,$$

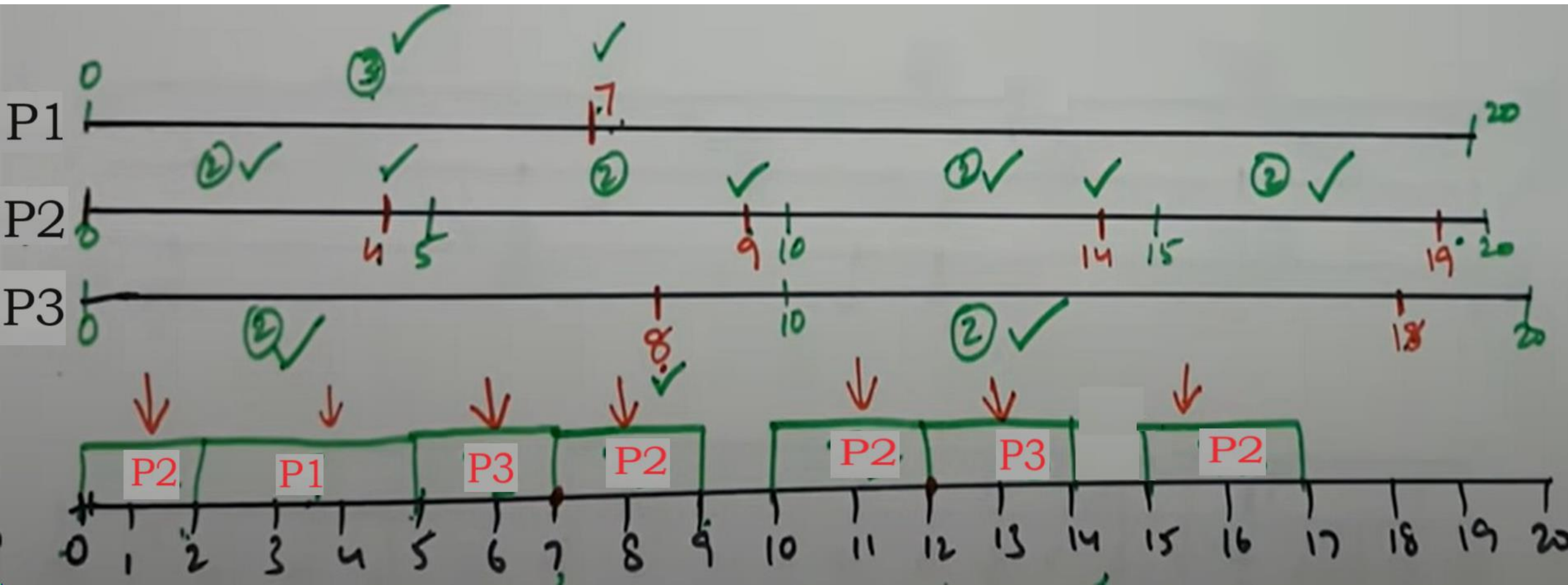
Where U is the CPU Utilization, C_i is the execution time, and T_i is the period for each process.

Process	Capacity	Deadline	Period
P1	3	7	20
P2	2	4	5
P3	2	8	10

We take the least common multiple of the periods of the processes to know how many units we need to execute the three processes:

$$\frac{L.C.M(20, 5, 10)}{20 \text{ units}}$$

We need 20 units at most to execute the processes.



- CPU utilization
 - Total time availability is 20
 - $P1 = 1 * 3 = 3$
 - $P2 = 4 * 2 = 8$
 - $P3 = 2 * 2 = 4$
 - Usability is $3 + 8 + 4 = 15$
 - CPU utilization is $15 / 20 = 75\%$

Points to remember

- **Hyper period**
- **Schedulability test Condition**
- **Closed deadline is Highest priority**
- **Preemption may occur**
- **Make sure that processor is executed only once within the period**
- **If not missed deadline**

Earliest Deadline First (EDF) scheduling –Preemption example

$T1 = (2, 0.9, 2)$, $T2 = (5, 2.3, 5)$ ----- Closest deadline ---highest priority

Task	Release time(r_i)	Execution Time(C_i)	Deadline (D_i)	Period(T_i)
T1	0	0.9	2	2
T2	0	2.3	5	5

Solve the following problems in the below link

<https://microcontrollerslab.com/earliest-deadline-first-scheduling/>

Given the following set of tasks:

Tasks	WCET	Period	Deadline
T1	1ms	3ms	3ms
T2	1ms	7ms	5ms
T3	2ms	20ms	19ms
T4	1ms	9ms	7ms

- (a) Given Earliest Deadline First (EDF) scheduling, is this set of tasks schedulable?
Show your work.
- (b) If schedulable, show a schedule that achieves this.

Consider the following four single-thread processes (P1 - P4) with the executing times, deadlines and periods shown in the table. Assume all processes arrive at timeslot 0. Fill in the table with the ID of the process that is running on the CPU with Rate-Monotonic (RM) scheduling and Earliest Deadline First (EDF) scheduling in the first 20 timeslots, and show how many deadlines are missed under each scheduler.

Process	Processing Time	Deadline	Period
P ₁	1	2	4
P ₂	1	5	6
P ₃	3	8	8
P ₄	2	7	15

Consider the following processes with their respective periods and execution times:

Process ID	Period	Execution time
P1	50	25
P2	75	25
P3	100	35

Can you find a schedule for these processes using the Earliest Deadline First (EDF) scheduling algorithm ?

- If yes, draw the Gantt chart and explain.
- If not, explain why ?

The Earliest Deadline First (EDF) algorithm is one of the scheduling algorithms used in operating systems.

	Process A	Process B	Process C	Process D
CPU cycles	3	6	2	4
Deadline	8	15	3	11

Using the information above, draw a time schedule showing how the four process will be executed. All processes are simultaneous, and their arrival time is the same. They all arrive at time Zero

Least slack time (LST) scheduling is an algorithm for dynamic priority scheduling. It assigns priorities to processes based on their slack time. Slack time is the amount of time left after a job if the job was started now. This algorithm is also known as least laxity first.

The task having minimum slack time will be considered the highest priority.

Least Slack Time scheduling algorithm works optimally only when preemption is allowed. It can produce a feasible schedule if and only if a feasible schedule exists for the set of tasks that are runnable.

<https://www.geeksforgeeks.org/least-slack-time-lst-scheduling-algorithm-in-real-time-systems/>

Rate-monotonic scheduling

What Is Priority Driven Scheduling

- Each process is assigned a priority.
- Process with highest priority is to be executed first and so on.
- Processes with same priority are executed on first come first served basis.
- It is having 2 types
 - Fixed
 - Dynamic

Rate monotonic scheduling

- **RMS** (Liu and Layland): widely-used, analyzable scheduling policy.
- Analysis is known as **Rate Monotonic Analysis (RMA)**.
- **RMA model**
 - All process run on single CPU.
 - Zero context switch time.
 - No data dependencies between processes.
 - Process execution time is constant.
 - Deadline is at end of period.
 - Highest-priority ready process runs.

- **Rate monotonic scheduling** is a priority algorithm that belongs to the static priority scheduling category of [Real Time Operating Systems](#).
- It is preemptive in nature.
- The priority is decided according to the cycle time of the processes that are involved.
- **If the process has a small job duration, then it has the highest priority.**
- Thus if a process with highest priority starts execution, it will preempt the other running processes.
- **The priority of a process is inversely proportional to the period it will run for.**

- A set of processes can be scheduled only if they satisfy the following equation:

$$U = \sum_{i=1}^n \frac{C_i}{T_i} \leq n(2^{1/n} - 1)$$

Where

- n is the number of processes in the process set,
- C_i is the computation time of the process,
- T_i is the Time period for the process to run and
- U is the processor utilization.

Example

Processes	Execution Time (C)	Time period (T)
P1	3	20
P2	2	5
P3	2	10

$$n(2^{1/n} - 1) = 3(2^{1/3} - 1) = 0.7977$$

$$U = 3/20 + 2/5 + 2/10 = 0.75$$

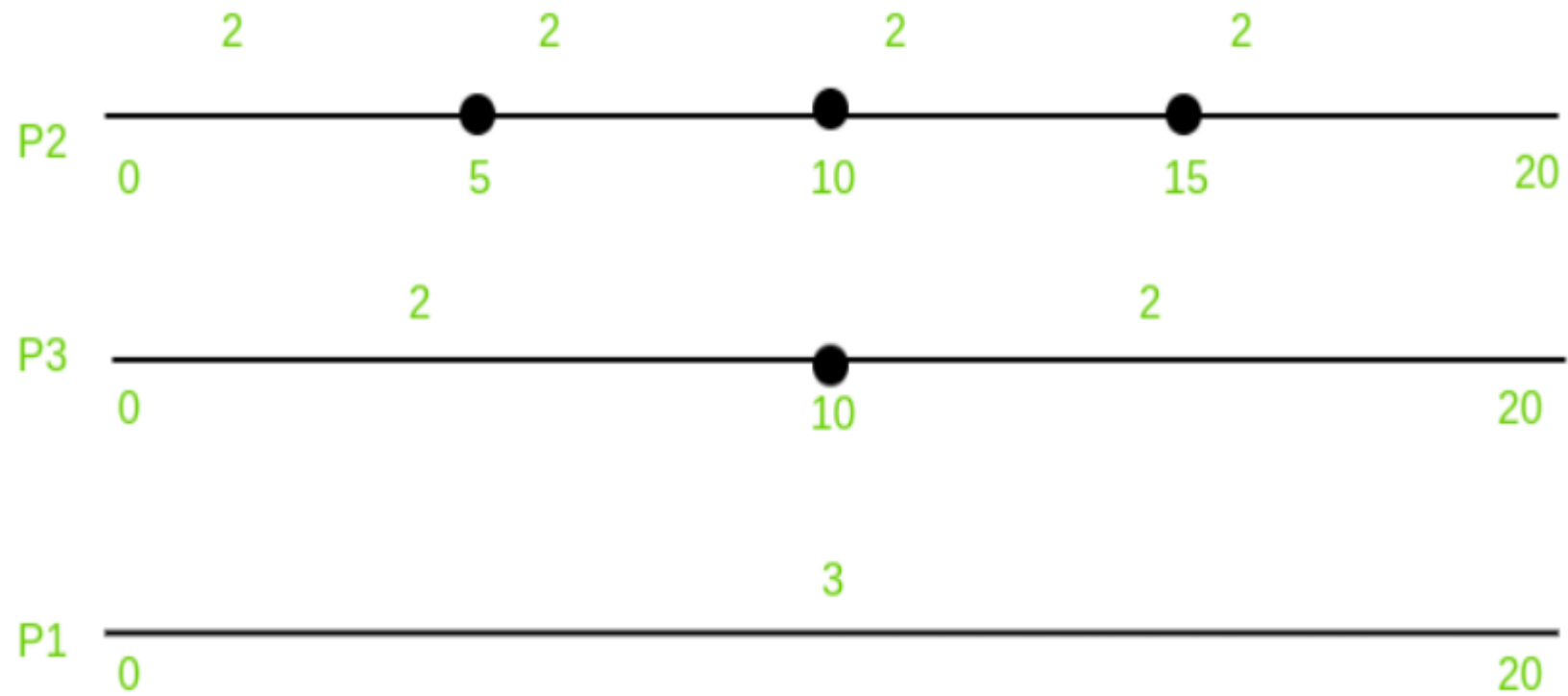
It is less than 1 or 100% utilization. The combined utilization of three processes is less than the threshold of these processes which means the above set of processes is schedulable and thus satisfies the above equation of the algorithm.

1.Scheduling time – For calculating the Scheduling time of the algorithm we have to take the LCM of the Time period of all the processes. LCM (20, 5, 10) of the above example is 20. Thus we can schedule it by 20-time units.

2.Priority – As discussed above, the priority will be the highest for the process which has the least running time period. Thus P2 will have the highest priority, and after that P3 and lastly P1.

$$P2 > P3 > P1$$

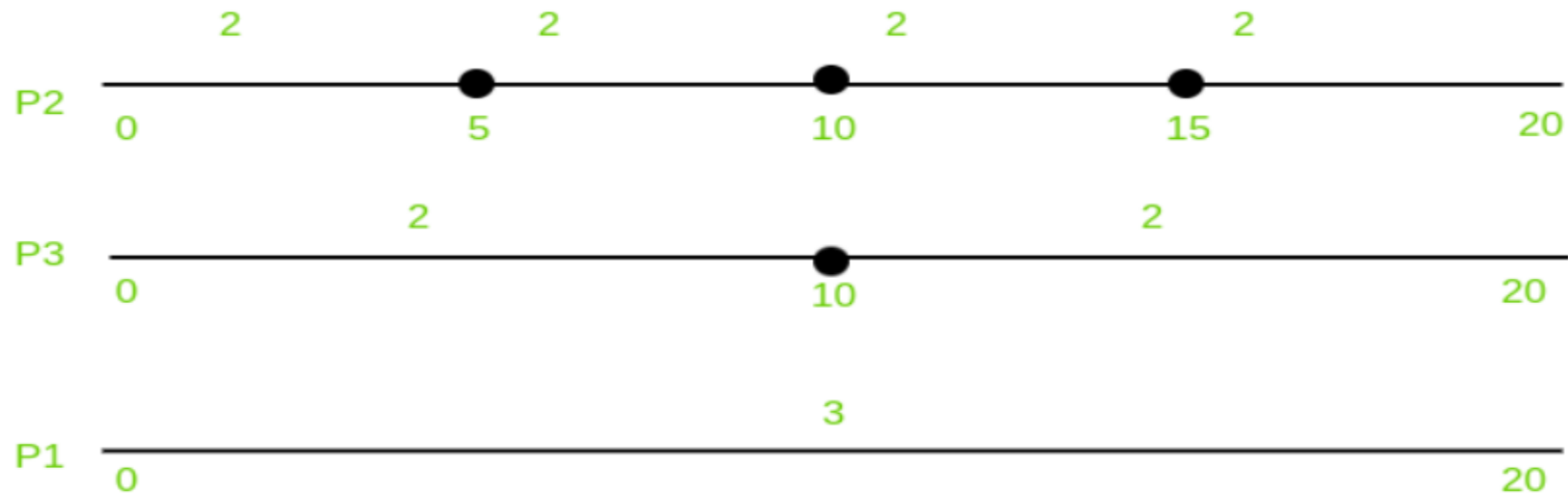
Representation and flow is as follows:



- The above figure says that Process P2 will execute two times for every 5 time units, Process P3 will execute two times for every 10 time units and Process P1 will execute three times in 20 time units. This has to be kept in mind for understanding the entire execution of the algorithm below.



Representation and flow is as follows:



Process	Capacity (C)	Period (P)
T1	3	20
T2	2	5
T3	2	10

a) For T2:

for every 5 period T2 has to execute 2 units

b) For T3:

for every 10 period T3 has to execute 2 units

c) For T1:

for every 20 period T1 has to execute 3 units

Process	Capacity (C)	Period (P)
T1	3	20
T2	2	5
T3	2	10

PRIORITY	PROCESS
1	T2
2	T3
3	T1

T1

T1 WILL EXECUTE 3 TIMES BETWEEN 1 TO 20 PERIODS

T3

T3 WILL EXECUTE 2 TIMES BETWEEN 1-10

T3 WILL EXECUTE 2 TIMES BETWEEN 10-20

T2

T2 WILL EXECUTE 2 TIME

T2 WILL EXECUTE 2 TIME

T2 WILL EXECUTE 2 TIME

T2 WILL EXECUTE 2 TIME

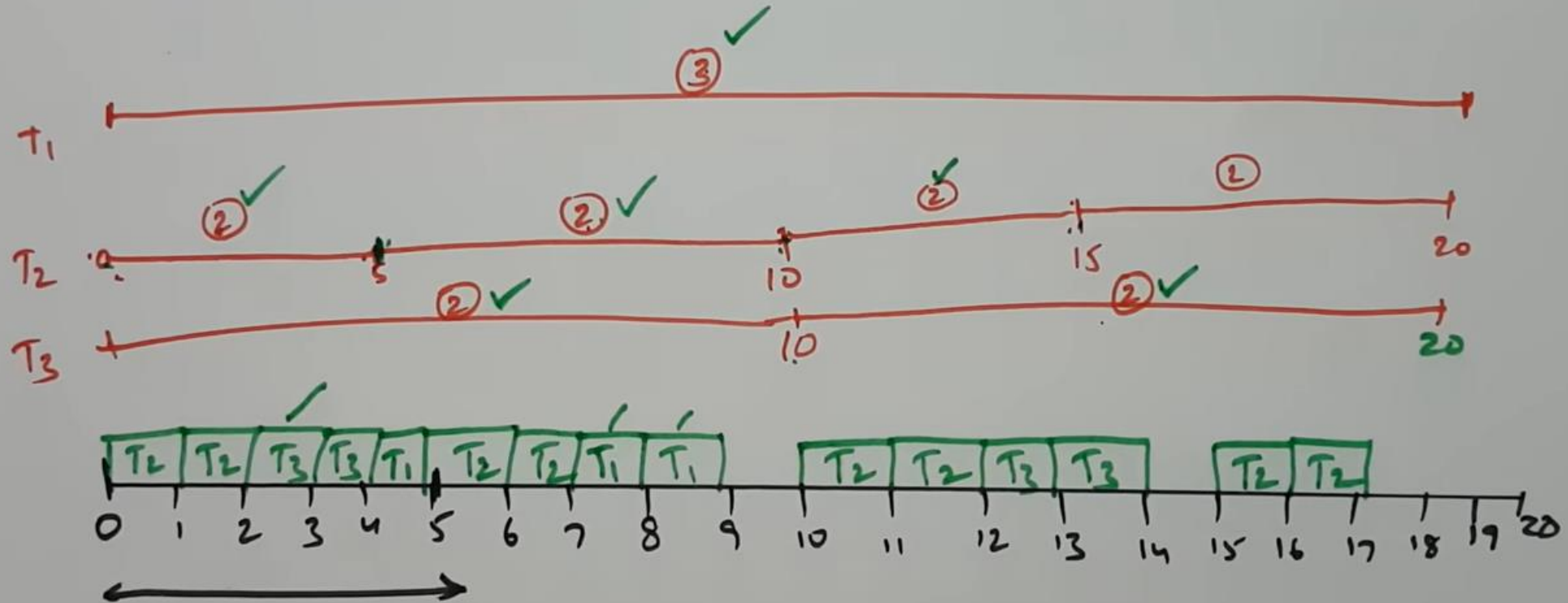
T2	T2	T3	T3	T1	T2	T2	T1	T1		T2	T2	T3	T3		T2	T2			
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20

Rate Monotonic Scheduling

	capacity	Period
T_1	3 (C)	20 (P)
→ T_2	2	5 ①
T_3	2	10 ②

① $L.C.M(20, 5, 10) = 20$

② Priority
 $T_2 > T_3 > T_1$
 ↑



- Process P2 will run first for 2 time units because it has the highest priority.
- After completing its two units, P3 will get the chance and thus it will run for 2 time units.
- As we know that process P2 will run 2 times in the interval of 5 time units and process P3 will run 2 times in the interval of 10 time units, they have fulfilled the criteria and thus now process P1 which has the least priority will get the chance and it will run for 1 time.
- And here the interval of five time units have completed.
- Because of its priority P2 will preempt P1 and thus will run 2 times.
- As P3 have completed its 2 time units for its interval of 10 time units, P1 will get chance and it will run for the remaining 2 times, completing its execution which was thrice in 20 time units.
- Now 9-10 interval remains idle as no process needs it.
- At 10 time units, process P2 will run for 2 times completing its criteria for the third interval (10-15).
- Process P3 will now run for two times completing its execution. Interval 14-15 will again remain idle for the same reason mentioned above.
- At 15 time unit, process P2 will execute for two times completing its execution.
- This is how the rate monotonic scheduling works.

Consider a uniprocessor system executing three tasks T1, T2 and T3, each of which is composed of an infinite sequence of jobs (or instances) which arrive periodically at intervals of 3, 7 and 20 milliseconds, respectively. The priority of each task is the inverse of its period and the available tasks are scheduled in order of priority, with the highest priority task scheduled first. Each instance of T1, T2 and T3 requires an execution time of 1, 2 and 4 milliseconds, respectively. Given that all tasks initially arrive at the beginning of the 1st milliseconds and task preemptions are allowed, the first instance of T3 completes its execution at the end of _____ milliseconds.

Mention all the arrival times for all the tasks

Draw the gantt chart

- (A) 5
- (B) 10
- (C) 12
- (D) 15

- **Conditions:**

The analysis of Rate monotonic scheduling assumes few properties that every process should possess. They are

- 1.Processes involved should not share the resources with other processes.
- 2.Deadlines must be similar to the time periods. Deadlines are deterministic.
- 3.Process running with highest priority that needs to run, will preempt all the other processes.
- 4.Priorities must be assigned to all the processes according to the protocol of Rate monotonic scheduling.

- CPU utilization
 - Total time availability is 20
 - $P1 = 1 * 3 = 3$
 - $P2 = 4 * 2 = 8$
 - $P3 = 2 * 2 = 4$
 - Usability is $3 + 8 + 4 = 15$
 - CPU utilization is $15 / 20 = 75\%$

<https://retis.sssup.it/~giorgio/paps/2005/rtsj05-rmedf.pdf>

<https://www.geeksforgeeks.org/difference-between-rate-monotonic-and-deadline-monotonic-scheduling/>

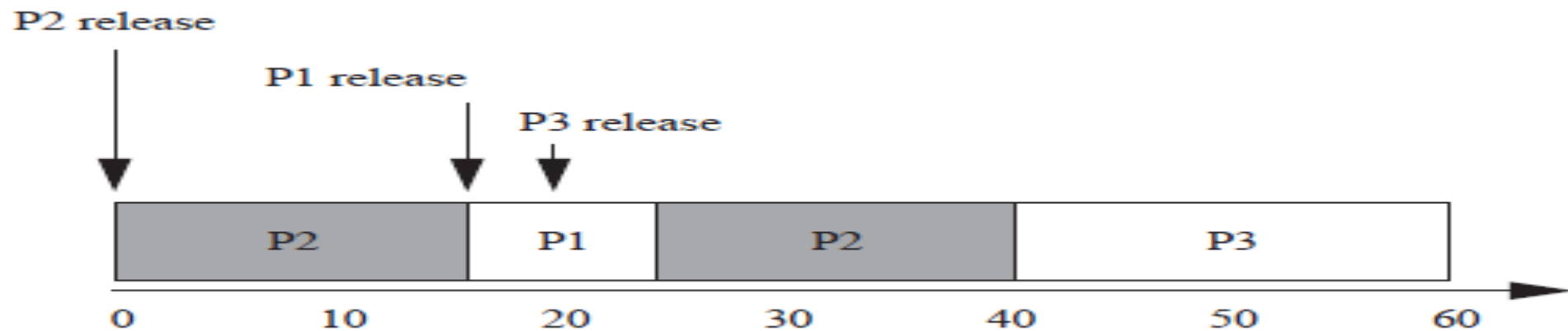
Round-robin scheduling

A common scheduling algorithm in general-purpose operating systems is roundrobin. All the processes are kept on a list and scheduled one after the other. This is generally combined with preemption so that one process does not grab all the CPU time. Round-robin scheduling provides a form of fairness in that all processes get a chance to execute. However, it does not guarantee the completion time of any task; as the number of processes increases, the response time of all the processes increases. Real-time systems, in contrast, require their notion of fairness to include timeliness and satisfaction of deadlines.

Priority-Driven Scheduling

A common way to choose the next executing process in an RTOS is based on process priorities. Each process is assigned a priority, an integer-valued number. The next process to be chosen to execute is the process in the set of ready processes that has the highest-valued priority. Example 6.1 shows how priorities can be used to schedule processes.

Process	Priority	Execution time
P1	1	10
P2	2	30
P3	3	20



For this example, we will adopt the following simple rules:

- Each process has a fixed priority that does not vary during the course of execution. (More sophisticated scheduling schemes do, in fact, change the priorities of processes to control what happens next.)
 - The ready process with the highest priority (with 1 as the highest priority of all) is selected for execution.
 - A process continues execution until it completes or it is preempted by a higher-priority process.
- We assume that P2 is ready to run when the system is started, P1's data arrive at time 15, and P3's data arrive at time 18. we can use the priorities to determine which process is running throughout the complete execution of the system.

When the system begins execution, P2 is the only ready process, so it is selected for execution. At time 15, P1 becomes ready; it preempts P2 and begins execution because it has a higher priority. Because P1 is the highest-priority process in the system, it is guaranteed to execute until it finishes. P3's data arrive at time 18, but it cannot preempt P1. Even when P1 finishes, P3 is not allowed to run. P2 is still ready and has higher priority than P3. Only after both P1 and P2 finish can P3 execute

